Data Visualization

Authors: Benedikt Prisett & Stijn Diemel

This notebook presents the visualizations that have been created to answer the provided questions. For each question we also include some of the earlier design iterations and then present the final visualization, that we ultimately used in the final multi-view dashboard. For the data cleaning process please view the notebook: 02_data_cleaning.ipynb.

In [1]:
```
import altair as alt
import pandas as pd
from pathlib import Path

_DIR = Path.cwd()

# Load clean dataset
df = pd.read_csv(_DIR / "data/clean_data/simpsons_episodes_clean.csv")
df["original_air_date"] = pd.to_datetime(df["original_air_date"])

# Derived columns
df["era"] = pd.cut(
    df["season"],
    bins=[0, 8, 18, 27],
    labels=["Golden Age (S1–8)", "Transition (S9–18)", "Decline (S19–27)"],
)

# Reusable Era Scales
era_domain = ["Golden Age (S1–8)", "Transition (S9–18)", "Decline (S19–27)"]
era_colors = ["#DDAA33", "#BB5566", "#004488", "#003f5c"]
era_scale = alt.Scale(domain=era_domain, range=era_colors)
```

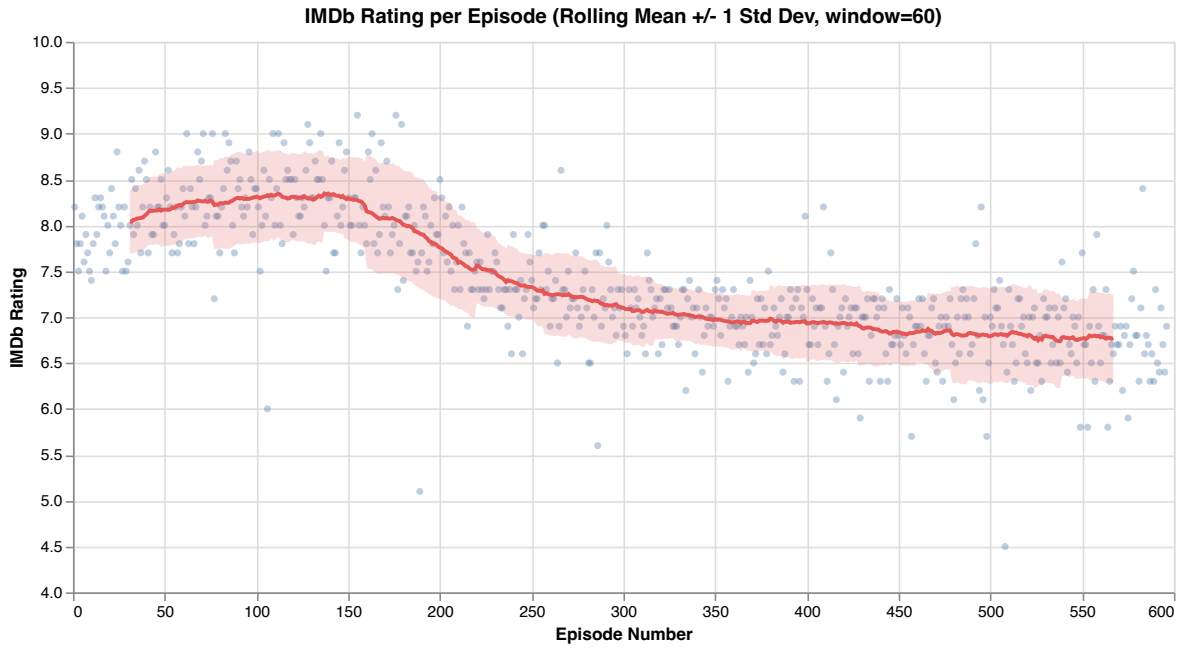
Question 1: How have the ratings evolved over time?

Q1: Design Iterations

In [2]:
```
# Scatter per episode + rolling mean with confidence band
window = 60

df_sorted = df.sort_values("number_in_series").copy()
df_sorted["rolling_mean_r"] = (
    df_sorted["imdb_rating"].rolling(window, center=True).mean()
)
df_sorted["rolling_std_r"] = df_sorted["imdb_rating"].rolling(window, center=True).st
df_sorted["rolling_upper_r"] = df_sorted["rolling_mean_r"] + df_sorted["rolling_std_r
df_sorted["rolling_lower_r"] = df_sorted["rolling_mean_r"] - df_sorted["rolling_std_r

q1_roll_dots = (
    alt.Chart(df_sorted)
    .mark_circle(size=20, opacity=0.35)
    .encode(
        x=alt.X("number_in_series:Q", title="Episode Number"),
        y=alt.Y(
            "imdb_rating:Q",
            title="IMDb Rating",
            scale=alt.Scale(domain=[4, 10]),
        ),
        color=alt.value("#4c78a8"),
        tooltip=["title:N", "season:O", "number_in_season:O", "imdb_rating:Q"],
```

```
    )
)

q1_roll_band = (
    alt.Chart(df_sorted)
    .mark_area(opacity=0.2, color="#e45756")
    .encode(
        x=alt.X("number_in_series:Q"),
        y=alt.Y("rolling_lower_r:Q"),
        y2=alt.Y2("rolling_upper_r:Q"),
    )
)

q1_roll_line = (
    alt.Chart(df_sorted)
    .mark_line(color="#e45756", strokeWidth=2.5)
    .encode(
        x=alt.X("number_in_series:Q"),
        y=alt.Y("rolling_mean_r:Q"),
    )
)

(q1_roll_dots + q1_roll_band + q1_roll_line).properties(
    width=700,
    height=350,
    title=f"IMDb Rating per Episode (Rolling Mean +/- 1 Std Dev, window={window})",
)
```

Out[2]:

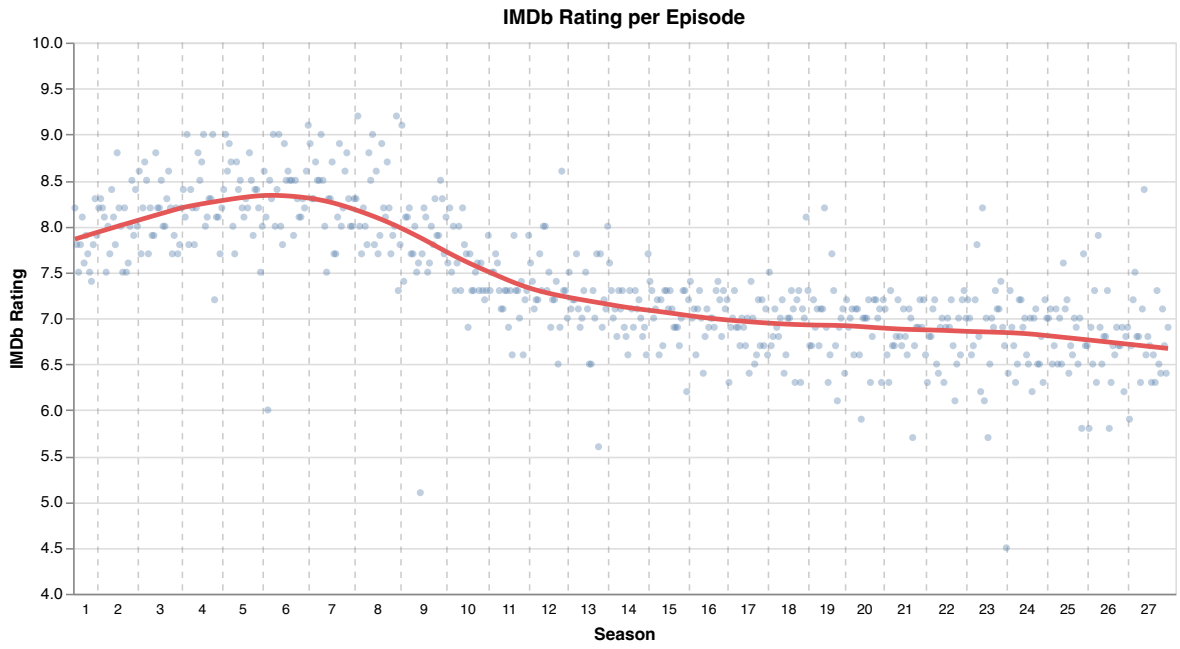


In [3]:

```
# Season boundaries: vertical lines between last ep of season N and first ep of seaso
season_bounds = (
    df.groupby("season")["number_in_series"].agg(["min", "max"]).reset_index()
)
season_dividers = pd.DataFrame(
    {
        "x": [
            (season_bounds.loc[i, "max"] + season_bounds.loc[i + 1, "min"]) / 2
            for i in range(len(season_bounds) - 1)
        ],
        "season_label": [str(s) for s in season_bounds["season"].iloc[:-1]],
    }
)

season_rules = (
    alt.Chart(season_dividers)
    .mark_rule(color="gray", strokeDash=[4, 4], opacity=0.4)
```

```
    .encode(x=alt.X("x:Q", axis=alt.Axis(grid=False)))
)

# Labels positioned at the midpoint of each season
season_midpoints = (
    df.groupby("season")["number_in_series"].agg(["min", "max"]).reset_index()
)
season_midpoints["mid"] = (season_midpoints["min"] + season_midpoints["max"]) / 2
season_midpoints["label"] = season_midpoints["season"].astype(str)

# LOESS with season numbers on x-axis (replacing episode numbers)
q1_loess_s_dots = (
    alt.Chart(df)
    .mark_circle(size=20, opacity=0.35)
    .encode(
        x=alt.X(
            "number_in_series:Q",
            title="Season",
            axis=alt.Axis(grid=False, labels=False, ticks=False, titlePadding=20),
        ),
        y=alt.Y(
            "imdb_rating:Q",
            title="IMDb Rating",
            scale=alt.Scale(domain=[4, 10]),
        ),
        color=alt.value("#4c78a8"),
        tooltip=["title:N", "season:O", "number_in_season:O", "imdb_rating:Q"],
    )
)

q1_loess_s_line = (
    alt.Chart(df)
    .transform_loess("number_in_series", "imdb_rating", bandwidth=0.2)
    .mark_line(color="#e45756", strokeWidth=3)
    .encode(
        x=alt.X("number_in_series:Q", axis=alt.Axis(grid=False)),
        y=alt.Y("imdb_rating:Q"),
    )
)

# Season number labels positioned just below the chart area
q1_season_x_labels = (
    alt.Chart(season_midpoints)
    .mark_text(fontSize=9, color="black")
    .encode(
        x=alt.X("mid:Q", axis=alt.Axis(grid=False)),
        y=alt.value(360),
        text="label:N",
    )
)

(q1_loess_s_dots + q1_loess_s_line + season_rules + q1_season_x_labels).properties(
    width=700,
    height=350,
    title="IMDb Rating per Episode",
)
```

Out[3]:

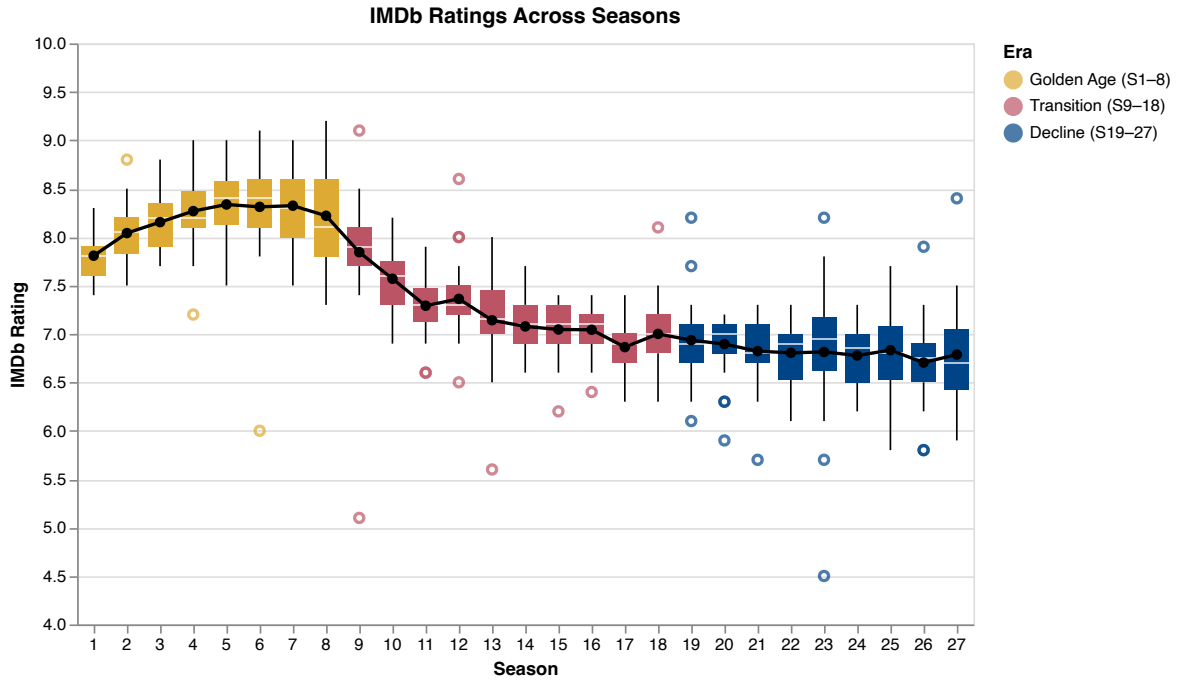


Q1: Final Visualization

In [4]:

```
q1_era_box = (
    alt.Chart(df)
    .mark_boxplot(size=15)
    .encode(
        x=alt.X("season:O", title="Season", axis=alt.Axis(labelAngle=0)),
        y=alt.Y("imdb_rating:Q", title="IMDb Rating", scale=alt.Scale(domain=[4, 10])),
        color=alt.Color("era:N", scale=era_scale, legend=alt.Legend(title="Era")),
    )
)
q1_era_mean = (
    alt.Chart(df)
    .mark_line(color="black", strokeWidth=2)
    .encode(x=alt.X("season:O"), y=alt.Y("mean(imdb_rating):Q"))
)
q1_era_points = q1_era_mean.mark_point(color="black", filled=True, size=40)
chart1 = (q1_era_box + q1_era_mean + q1_era_points).properties(
    height=350, title="IMDb Ratings Across Seasons"
)
chart1
```

Out[4]:



To answer how ratings have evolved over time, we chose box plots per season overlaid with a connected mean line. The box plots show the full distribution of episode ratings within each season, making it easy to spot both the central tendency and the spread. The mean line connects season averages to reveal the overall trend at a glance and is colored black to separate it visually from the colored distribution boxes. In previous iterations, we explored scatter plots showing all episode ratings with an overlaid rolling mean and standard deviation as well as a LOESS regression overlay. We decided against these approaches, as scatter plots show a level of detail not required to answer how ratings evolved over time and can quickly become overwhelming for the viewer. To improve legibility, we used short, horizontal axis labels and trimmed the y-axis from 4 to 10, since no ratings fall outside this range, allowing differences to be seen more clearly. We also applied a coloring scheme that divides the series into three distinct eras. This design decision was introduced while working on the correlation question and the combined multi-view dashboard, and will be elaborated on in a later section.

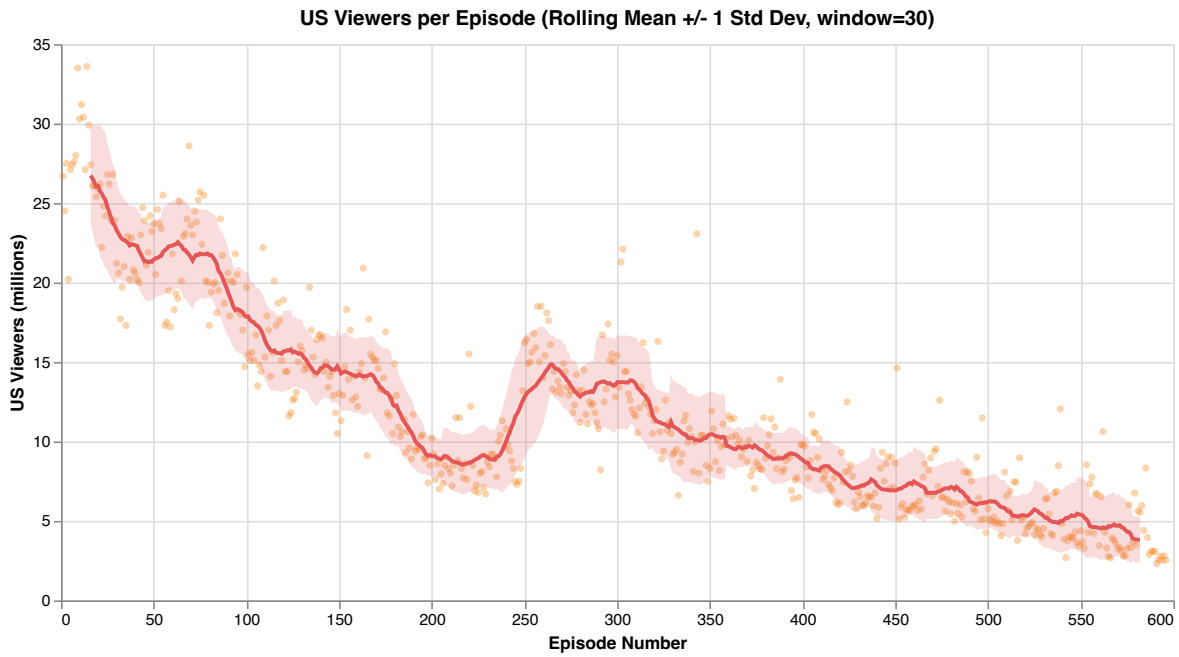
Question 2: How have the viewers evolved over time?

Q2: Design Iterations

In [5]:
```
# Scatter per episode + rolling mean with confidence band
window = 30

df_sorted_v = df.sort_values("number_in_series").copy()
df_sorted_v["rolling_mean_v"] = (
    df_sorted_v["us_viewers_in_millions"].rolling(window, center=True).mean()
)
df_sorted_v["rolling_std_v"] = (
    df_sorted_v["us_viewers_in_millions"].rolling(window, center=True).std()
)
df_sorted_v["rolling_upper_v"] = (
    df_sorted_v["rolling_mean_v"] + df_sorted_v["rolling_std_v"]
)
df_sorted_v["rolling_lower_v"] = (
    df_sorted_v["rolling_mean_v"] - df_sorted_v["rolling_std_v"]
)

q2_roll_dots = (
    alt.Chart(df_sorted_v)
    .mark_circle(size=20, opacity=0.35)
    .encode(
        x=alt.X("number_in_series:Q", title="Episode Number"),
        y=alt.Y("us_viewers_in_millions:Q", title="US Viewers (millions)"),
        color=alt.value("#f58518"),
        tooltip=[
            "title:N",
            "season:O",
            "number_in_season:O",
            "us_viewers_in_millions:Q",
        ],
    )
)

q2_roll_band = (
    alt.Chart(df_sorted_v)
    .mark_area(opacity=0.2, color="#e45756")
    .encode(
        x=alt.X("number_in_series:Q"),
        y=alt.Y("rolling_lower_v:Q"),
        y2=alt.Y2("rolling_upper_v:Q"),
```

```
    )
)

q2_roll_line = (
    alt.Chart(df_sorted_v)
    .mark_line(color="#e45756", strokeWidth=2.5)
    .encode(
        x=alt.X("number_in_series:Q"),
        y=alt.Y("rolling_mean_v:Q"),
    )
)

(q2_roll_dots + q2_roll_band + q2_roll_line).properties(
    width=700,
    height=350,
    title="US Viewers per Episode (Rolling Mean +/- 1 Std Dev, window=30)",
)
```

Out[5]:

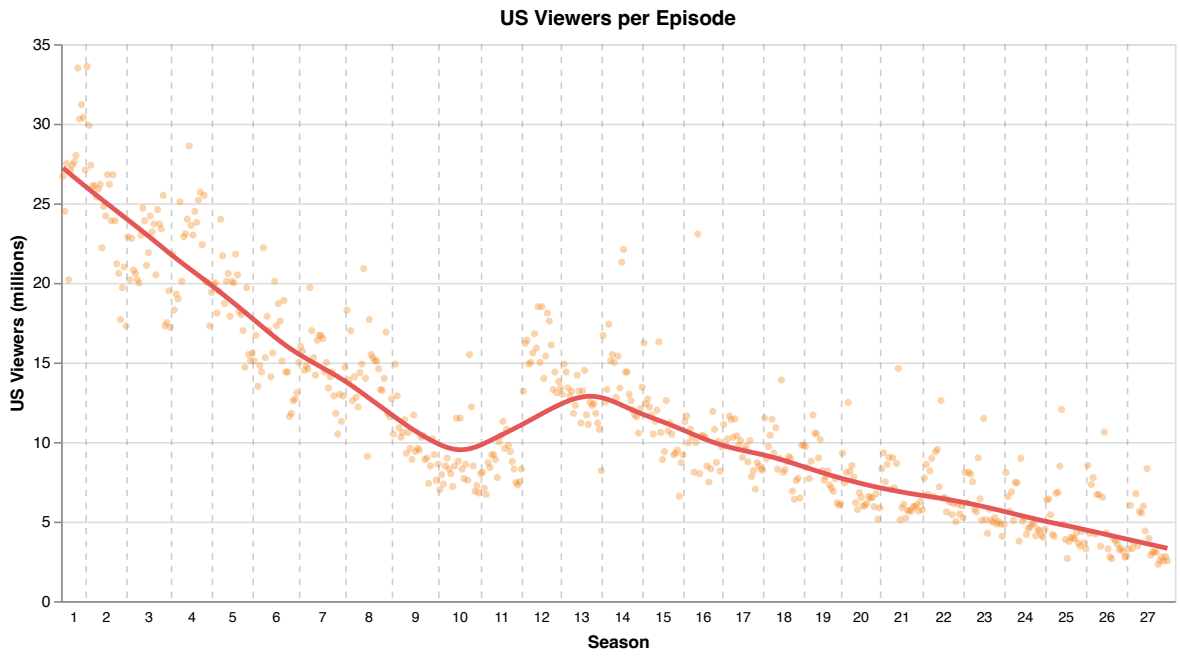


In [6]:

```
# LOESS with season numbers on x-axis (replacing episode numbers)
q2_loess_s_dots = (
    alt.Chart(df)
    .mark_circle(size=20, opacity=0.35)
    .encode(
        x=alt.X(
            "number_in_series:Q",
            title="Season",
            axis=alt.Axis(grid=False, labels=False, ticks=False, titlePadding=20),
        ),
        y=alt.Y("us_viewers_in_millions:Q", title="US Viewers (millions)"),
        color=alt.value("#f58518"),
        tooltip=[
            "title:N",
            "season:O",
            "number_in_season:O",
            "us_viewers_in_millions:Q",
        ],
    )
)

q2_loess_s_line = (
    alt.Chart(df)
    .transform_loess("number_in_series", "us_viewers_in_millions", bandwidth=0.2)
    .mark_line(color="#e45756", strokeWidth=3)
    .encode(
        x=alt.X("number_in_series:Q", axis=alt.Axis(grid=False)),
```

```
        y=alt.Y("us_viewers_in_millions:Q"),
    )
)

q2_season_x_labels = (
    alt.Chart(season_midpoints)
    .mark_text(fontSize=9, color="black")
    .encode(
        x=alt.X("mid:Q", axis=alt.Axis(grid=False)),
        y=alt.value(360),
        text="label:N",
    )
)

(q2_loess_s_dots + q2_loess_s_line + season_rules + q2_season_x_labels).properties(
    width=700, height=350, title="US Viewers per Episode"
)
```

Out[6]:

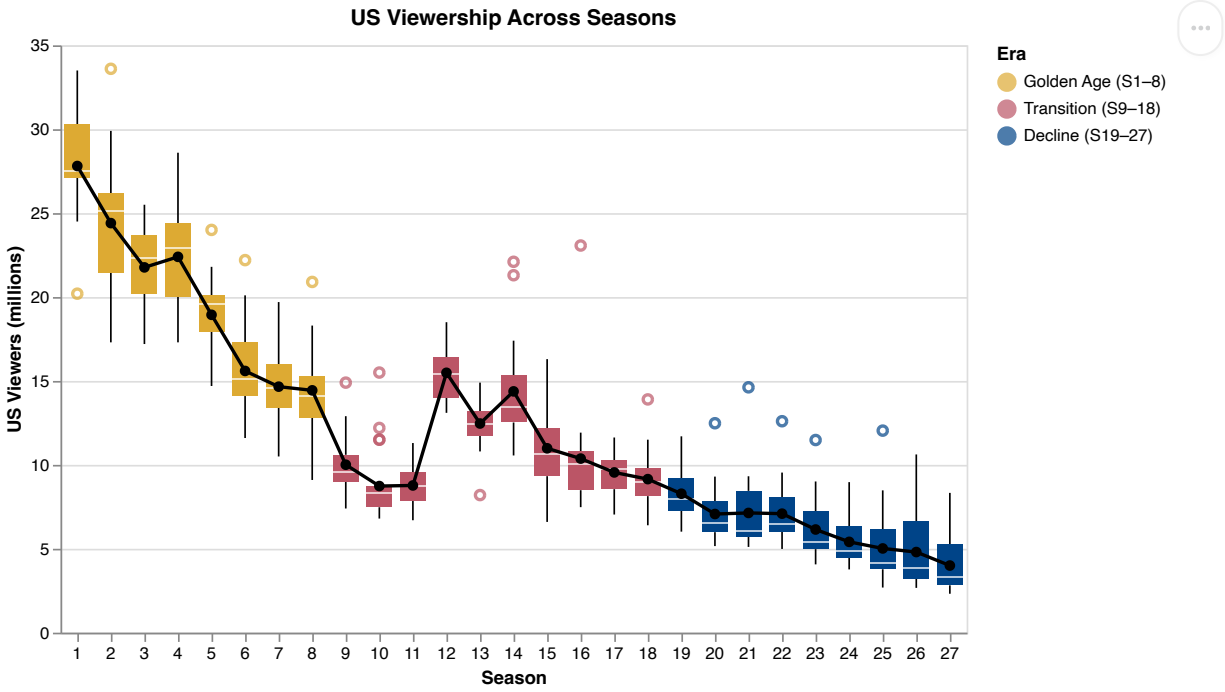


Q2: Final Visualization

In [7]:

```
q2_era_box = (
    alt.Chart(df)
    .mark_boxplot(size=15)
    .encode(
        x=alt.X("season:O", title="Season", axis=alt.Axis(labelAngle=0)),
        y=alt.Y("us_viewers_in_millions:Q", title="US Viewers (millions)"),
        color=alt.Color("era:N", scale=era_scale, legend=alt.Legend(title="Era")),
    )
)
q2_era_mean = (
    alt.Chart(df)
    .mark_line(color="black", strokeWidth=2)
    .encode(x=alt.X("season:O"), y=alt.Y("mean(us_viewers_in_millions):Q"))
)
q2_era_points = q2_era_mean.mark_point(color="black", filled=True, size=40)
chart2 = (q2_era_box + q2_era_mean + q2_era_points).properties(
    height=350, title="US Viewership Across Seasons"
)
chart2
```

Out[7]:



For the second question we used a similar visualization style as the previous one, opting for box plots per season colored by era overlaid with a black connected mean line. This choice was intentional, as the question follows the same style and intention as the first and using a consistent visual language allows direct comparison with the ratings chart. Unlike chart 1, where we trimmed the y-axis, here we kept the natural axis range since viewer counts span a much wider and drop significantly for later seasons. As with the ratings chart, we considered scatter plots with rolling means and LOESS overlays but rejected them for the same reasons. A person reading this chart can follow the mean line to see that viewership peaked in the early seasons before entering a steady decline, while the box plots reveal that early seasons also exhibited much wider within-season variance.

Question 3: Is there a correlation between the gradings and the viewers?

Q3: Design Iterations

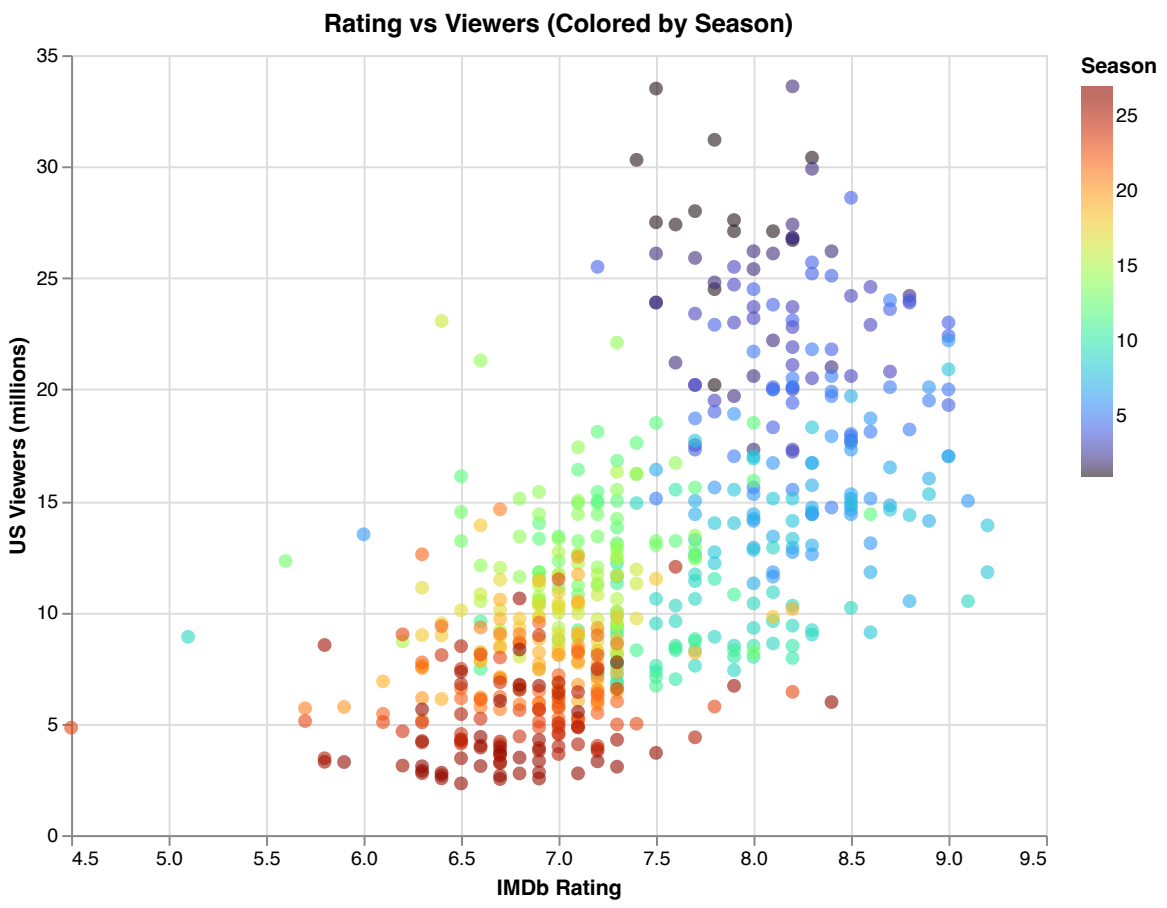
In [8]:
```
# Scatter + single regression line
q3d_scatter = (
    alt.Chart(df)
    .mark_circle(size=40, opacity=0.4)
    .encode(
        x=alt.X("imdb_rating:Q", title="IMDb Rating", scale=alt.Scale(zero=False)),
        y=alt.Y("us_viewers_in_millions:Q", title="US Viewers (millions)"),
        color=alt.value("#4c78a8"),
        tooltip=["title:N", "season:O", "imdb_rating:Q", "us_viewers_in_millions:Q"],
    )
)

q3d_reg = (
    q3d_scatter.transform_regression("imdb_rating", "us_viewers_in_millions")
    .mark_line(color="#e45756", strokeWidth=2)
    .encode(color=alt.value("#e45756"))
)

(q3d_scatter + q3d_reg).properties(
    width=500, height=400, title="Rating vs Viewers (with Regression Line)"
)
```

Out[8]:

In [9]:

```
# Scatter colored by season (all 27)
alt.Chart(df).mark_circle(size=50, opacity=0.6).encode(
    x=alt.X("imdb_rating:Q", title="IMDb Rating", scale=alt.Scale(zero=False)),
    y=alt.Y("us_viewers_in_millions:Q", title="US Viewers (millions)"),
    color=alt.Color("season:Q", scale=alt.Scale(scheme="turbo"), title="Season"),
    tooltip=["title:N", "season:O", "imdb_rating:Q", "us_viewers_in_millions:Q"],
).properties(width=500, height=400, title="Rating vs Viewers (Colored by Season)")
```

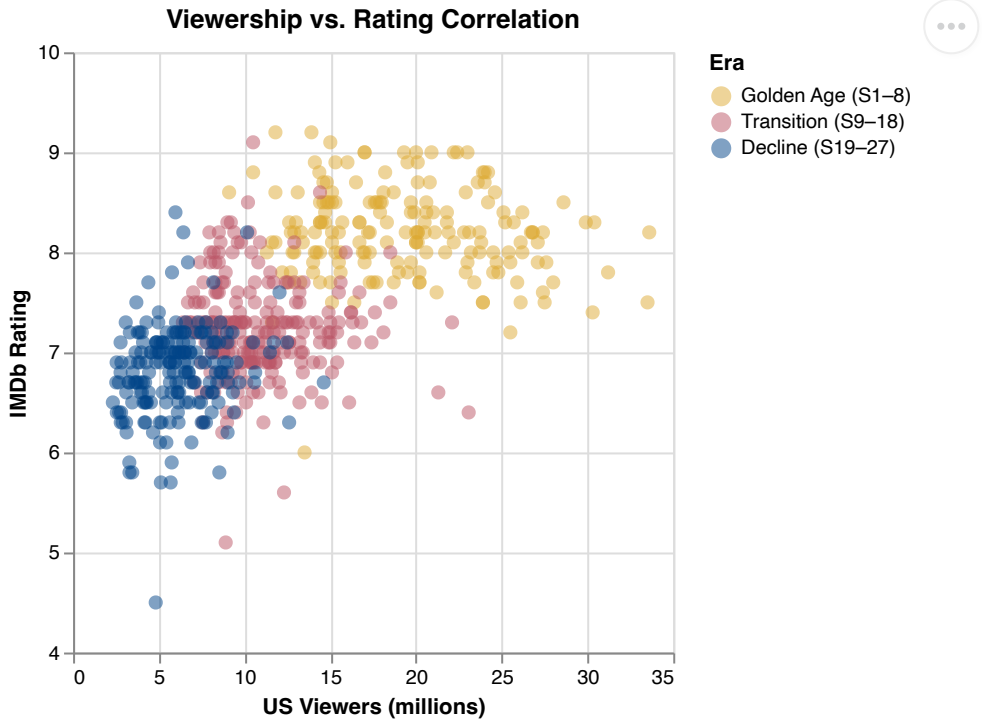
Out[9]:



Q3: Final Visualization

In [10]:
```
chart3 = (
    alt.Chart(df)
    .mark_circle(size=50, opacity=0.5)
    .encode(
        x=alt.X("us_viewers_in_millions:Q", title="US Viewers (millions)"),
        y=alt.Y("imdb_rating:Q", title="IMDb Rating", scale=alt.Scale(domain=[4, 10])),
        color=alt.Color("era:N", scale=era_scale, legend=alt.Legend(title="Era")),
        tooltip=[
            "title:N",
            "season:O",
            "era:N",
            "imdb_rating:Q",
            "us_viewers_in_millions:Q",
        ],
    )
    .properties(height=300, title="Viewership vs. Rating Correlation")
)
chart3
```

Out[10]:



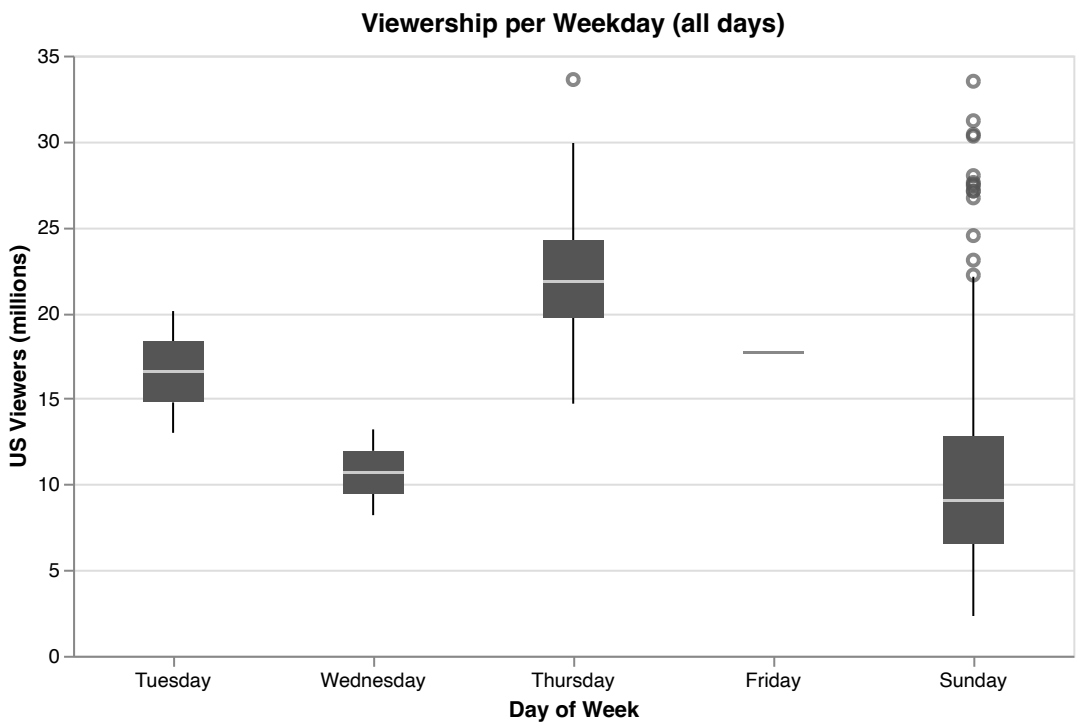
We chose a scatter plot as it's the natural choice for showing the relationship between two quantitative variables. A reader can see that the point cloud slopes upward, suggesting that episodes with higher viewership tend to receive higher ratings. In earlier iterations we included a trend line, but the positive correlation is visible from the cloud alone, making it redundant. We also tested a continuous color scale encoding each season individually, however distinguishing 27 colors proved impractical. Nevertheless this revealed an interesting pattern: early seasons cluster in the high-viewership, high-rating region while later seasons drift toward the lower end of both axes. This observation directly inspired the three-era grouping, with boundaries chosen to create roughly equal groups that align with visible drops in the first two charts. This color scheme makes the clustering immediately visible and highlights our observation. The axes were initially flipped, but we reversed this because viewership is measured at premiere and is not directly influenced by ratings made afterwards. Following convention, the more independent variable (viewership) is placed on the x-axis and the dependent (rating) on the y-axis. For legibility, we used semi-transparent points to reveal overlapping data in dense regions and trimmed both axes to the relevant data range.

Question 4: Are the number of viewers for the episodes related to the weekday they were aired?

Q4: Design Iterations

In [11]:
```
weekday_order = [
    "Monday",
    "Tuesday",
    "Wednesday",
    "Thursday",
    "Friday",
    "Saturday",
    "Sunday",
]
chart4_all = (
    alt.Chart(df)
    .mark_boxplot(size=30)
    .encode(
        x=alt.X(
            "weekday:O",
            sort=weekday_order,
            title="Day of Week",
            axis=alt.Axis(labelAngle=0),
        ),
        y=alt.Y("us_viewers_in_millions:Q", title="US Viewers (millions)"),
        color=alt.Color(
            "weekday:N",
            sort=weekday_order,
            legend=None,
            scale=alt.Scale(range=["#555555"]),
        ),
    )
    .properties(width=500, height=300, title="Viewership per Weekday (all days)")
)
chart4_all
```

Out[11]:



In [12]:
```
# Weekday distribution per season
weekday_order = [
    "Monday",
    "Tuesday",
```

```
    "Wednesday",
    "Thursday",
    "Friday",
    "Saturday",
    "Sunday",
]
ct = pd.crosstab(df["season"], df["weekday"]).reindex(
    columns=weekday_order, fill_value=0
)
ct["Total"] = ct.sum(axis=1)
print(ct.to_string())

print("\nSummary:")
for day in weekday_order:
    n = (df["weekday"] == day).sum()
    if n > 0:
        seasons = sorted(
            int(s) for s in df.loc[df["weekday"] == day, "season"].unique()
        )
        print(f"  {day}: {n} episodes (seasons {seasons})")
```

weekday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday	Total
season								
1	0	0	0	0	0	0	13	13
2	0	0	0	22	0	0	0	22
3	0	0	0	24	0	0	0	24
4	0	1	0	21	0	0	0	22
5	0	0	0	22	0	0	0	22
6	0	0	0	0	0	0	25	25
7	0	0	0	0	0	0	25	25
8	0	0	0	0	1	0	24	25
9	0	0	0	0	0	0	25	25
10	0	0	0	0	0	0	23	23
11	0	0	0	0	0	0	22	22
12	0	0	1	0	0	0	20	21
13	0	1	1	0	0	0	20	22
14	0	0	0	0	0	0	22	22
15	0	0	0	0	0	0	22	22
16	0	0	0	0	0	0	21	21
17	0	0	0	0	0	0	22	22
18	0	0	0	0	0	0	22	22
19	0	0	0	0	0	0	20	20
20	0	0	0	0	0	0	21	21
21	0	0	0	0	0	0	23	23
22	0	0	0	0	0	0	22	22
23	0	0	0	0	0	0	22	22
24	0	0	0	0	0	0	22	22
25	0	0	0	0	0	0	22	22
26	0	0	0	0	0	0	22	22
27	0	0	0	0	0	0	22	22

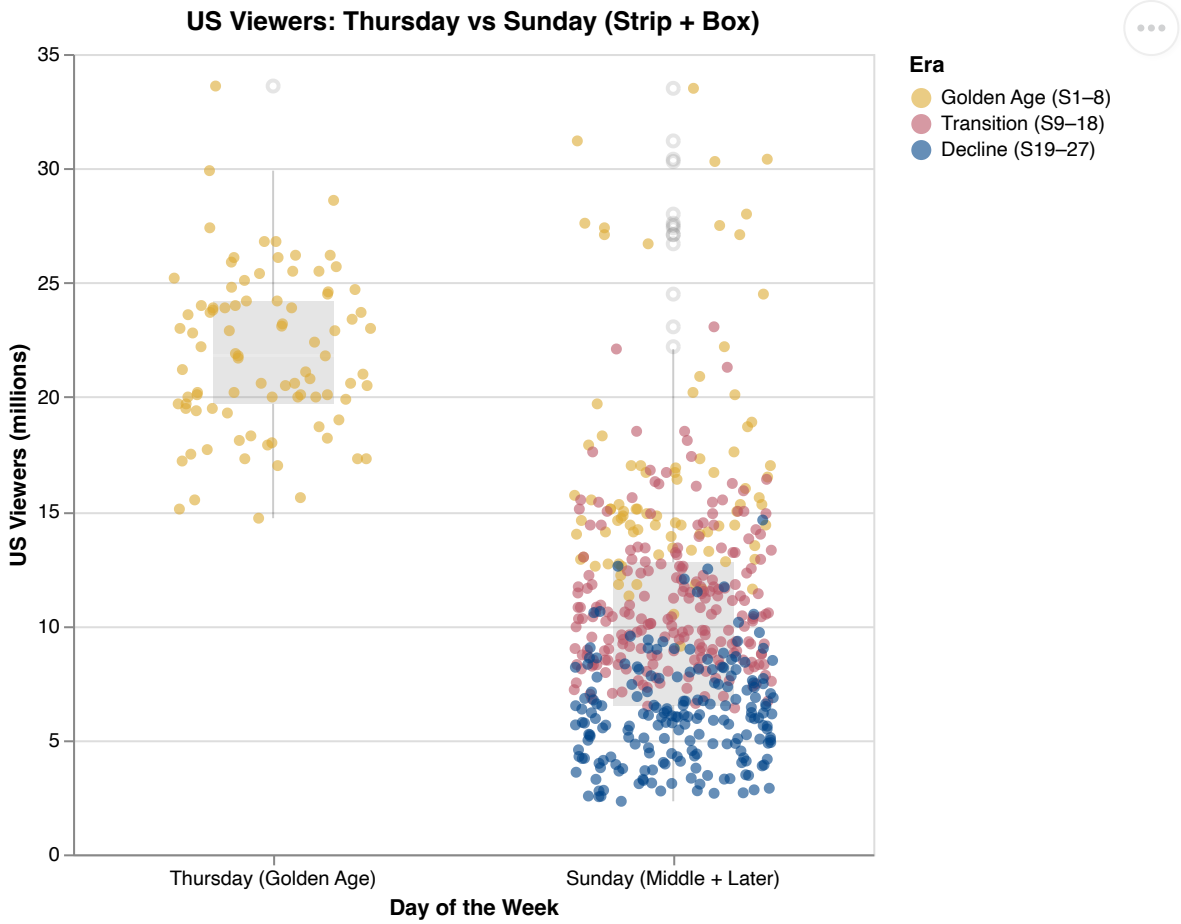
```
Summary:
  Tuesday: 2 episodes (seasons [4, 13])
  Wednesday: 2 episodes (seasons [12, 13])
  Thursday: 89 episodes (seasons [2, 3, 4, 5])
  Friday: 1 episodes (seasons [8])
  Sunday: 502 episodes (seasons [1, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 1
9, 20, 21, 22, 23, 24, 25, 26, 27])
```

In [13]:

```
import numpy as np

# Strip + box plot combined (Thu vs Sun), dots colored by era
df_thu_sun = df[df["weekday"].isin(["Thursday", "Sunday"])].copy()
thu_sun_labels = ["Thursday\n(Golden Age)", "Sunday\n(Middle + Later)"]
df_thu_sun["weekday_label"] = df_thu_sun["weekday"].map(
    {"Thursday": thu_sun_labels[0], "Sunday": thu_sun_labels[1]}
```

```
)

np.random.seed(42)
df_thu_sun["jitter_offset"] = np.random.uniform(-0.2, 0.2, len(df_thu_sun))

q4e_box = (
    alt.Chart(df_thu_sun)
    .mark_boxplot(size=60, opacity=0.2)
    .encode(
        x=alt.X(
            "weekday_label:N",
            sort=thu_sun_labels,
            title="Day of the Week",
            axis=alt.Axis(labelAngle=0),
        ),
        y=alt.Y("us_viewers_in_millions:Q", title="US Viewers (millions)"),
        color=alt.value("gray"),
    )
)

q4e_dots = (
    alt.Chart(df_thu_sun)
    .mark_circle(size=30, opacity=0.6)
    .encode(
        x=alt.X(
            "weekday_label:N",
            sort=thu_sun_labels,
            title="Day of the Week",
            axis=alt.Axis(labelAngle=0),
        ),
        xOffset=alt.XOffset(
            "jitter_offset:Q",
            scale=alt.Scale(domain=[-0.4, 0.4]),
        ),
        y=alt.Y("us_viewers_in_millions:Q", title="US Viewers (millions)"),
        color=alt.Color(
            "era:N",
            scale=era_scale,
            title="Era",
        ),
        tooltip=[
            "title:N",
            "season:O",
            "era:N",
            "weekday:N",
            "us_viewers_in_millions:Q",
        ],
    )
)

(q4e_box + q4e_dots).properties(
    width=400, height=400, title="US Viewers: Thursday vs Sunday (Strip + Box)"
)
```

Out[13]:

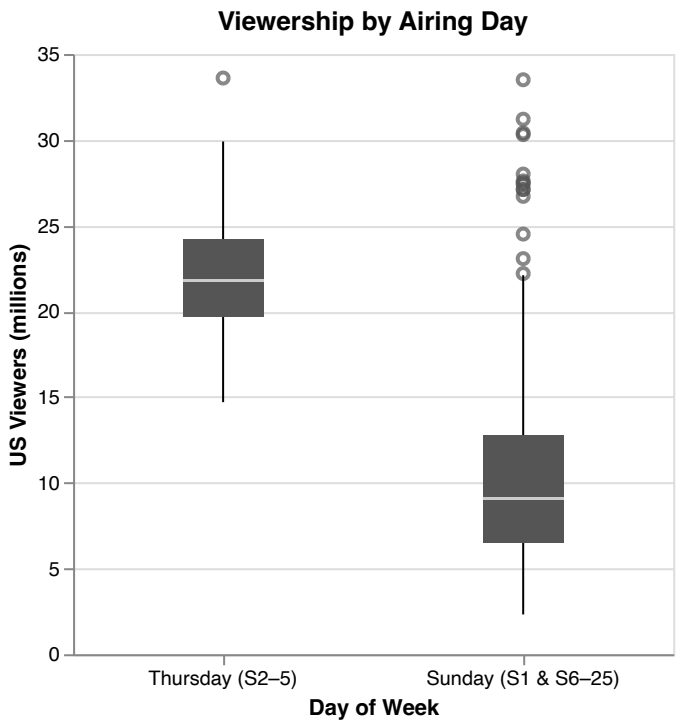


Q4: Final Visualization

In [14]:

```
df_thu_sun = df[df["weekday"].isin(["Thursday", "Sunday"])].copy()
thu_sun_labels = ["Thursday (S2–5)", "Sunday (S1 & S6–25)"]
df_thu_sun["weekday_label"] = df_thu_sun["weekday"].map(
    {"Thursday": thu_sun_labels[0], "Sunday": thu_sun_labels[1]}
)
chart4 = (
    alt.Chart(df_thu_sun)
    .mark_boxplot(size=40)
    .encode(
        x=alt.X(
            "weekday_label:O",
            sort=thu_sun_labels,
            title="Day of Week",
            axis=alt.Axis(labelAngle=0, labelAlign="center"),
        ),
        y=alt.Y("us_viewers_in_millions:Q", title="US Viewers (millions)"),
        color=alt.Color(
            "weekday_label:N",
            sort=thu_sun_labels,
            legend=None,
            scale=alt.Scale(domain=thu_sun_labels, range=["#555555"]),
        ),
    )
    .properties(width=300, height=300, title="Viewership by Airing Day")
)
chart4
```

Out[14]:



To assess how the number of viewers is related to the airing weekday, we opted for side-by-side box plots comparing Thursday and Sunday episodes. A reader can see at a glance that Thursday episodes drew substantially higher viewership with less spread, though this largely reflects that Thursday episodes come from the early, high-viewership seasons. We initially created box plots for all weekdays, but the resulting chart showed unusual patterns for several days. After inspecting the underlying data, we confirmed that the dataset is overwhelmingly split between Thursdays and Sundays, with only two episodes on Tuesday, two on Wednesday, and one on Friday. We dropped these days as outliers that do not meaningfully contribute to answering the question. We considered overlaying the box plots with a jittered strip plot to visually show that only early-era episodes aired on Thursdays. However, the scattered points made the boxes harder to read and the chart more cluttered. We instead decided to focus on the essentials: two clean box plots, with season ranges included in the x-axis labels (e.g., "Thursday (S2–5)") to alert the reader that the viewership difference may also be related to the era rather than the weekday alone.

5: Do the seasons' number of viewers present any relevant pattern?

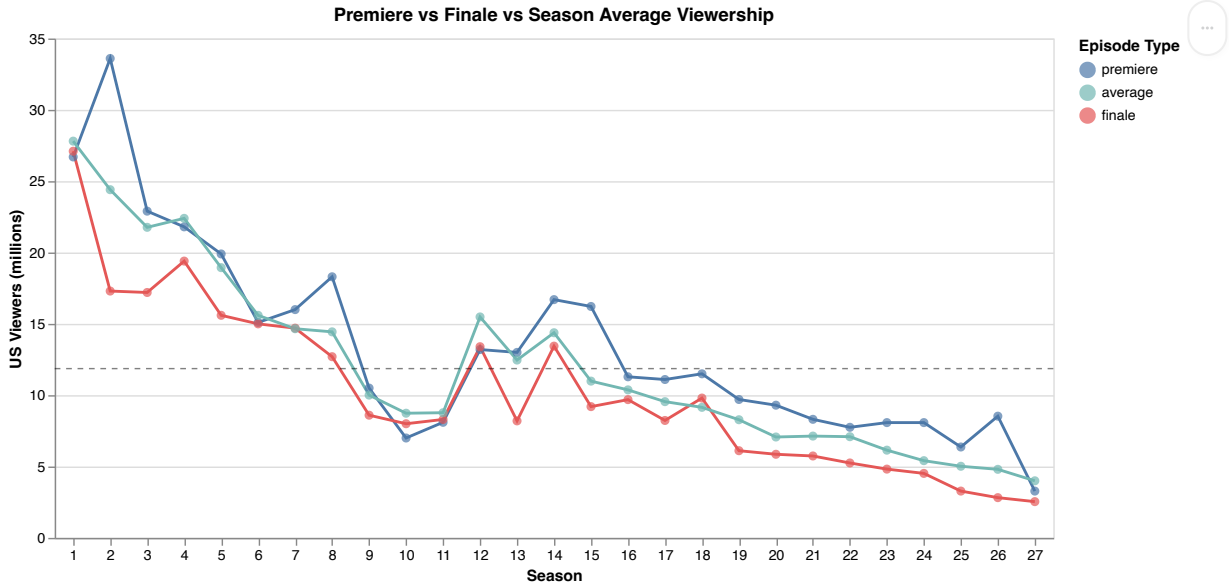
Q5: Design Iterations

In [15]:

```
# Premiere vs Finale vs Season Average
season_stats = (
    df.groupby("season")
    .agg(
        premiere=("us_viewers_in_millions", "first"),
        finale=("us_viewers_in_millions", "last"),
        average=("us_viewers_in_millions", "mean"),
    )
    .reset_index()
)

# Reshape to long format for plotting
season_long = season_stats.melt(
    id_vars="season",
    value_vars=["premiere", "finale", "average"],
    var_name="type",
    value_name="us_viewers_in_millions",
```

```
)

type_domain = ["premiere", "average", "finale"]
type_colors = ["#4c78a8", "#72b7b2", "#e45756"]

q5_lines = (
    alt.Chart(season_long)
    .mark_line(strokeWidth=2)
    .encode(
        x=alt.X("season:O", title="Season", axis=alt.Axis(labelAngle=0)),
        y=alt.Y("us_viewers_in_millions:Q", title="US Viewers (millions)"),
        color=alt.Color(
            "type:N",
            scale=alt.Scale(domain=type_domain, range=type_colors),
            title="Episode Type",
        ),
    )
)

q5_points = (
    alt.Chart(season_long)
    .mark_point(filled=True, size=40)
    .encode(
        x=alt.X("season:O", axis=alt.Axis(labelAngle=0)),
        y=alt.Y("us_viewers_in_millions:Q"),
        color=alt.Color(
            "type:N",
            scale=alt.Scale(domain=type_domain, range=type_colors),
            title="Episode Type",
        ),
        tooltip=[
            alt.Tooltip("season:O", title="Season"),
            alt.Tooltip("type:N", title="Type"),
            alt.Tooltip("us_viewers_in_millions:Q", title="Viewers (M)", format=".2f"
        ],
    )
)

# Overall series mean
overall_mean = df["us_viewers_in_millions"].mean()
q5_mean_rule = (
    alt.Chart(pd.DataFrame({"y": [overall_mean]}))
    .mark_rule(color="black", strokeDash=[4, 4], opacity=0.5)
    .encode(y="y:Q")
)

(q5_lines + q5_points + q5_mean_rule).properties(
    width=700, height=350, title="Premiere vs Finale vs Season Average Viewership"
)
```

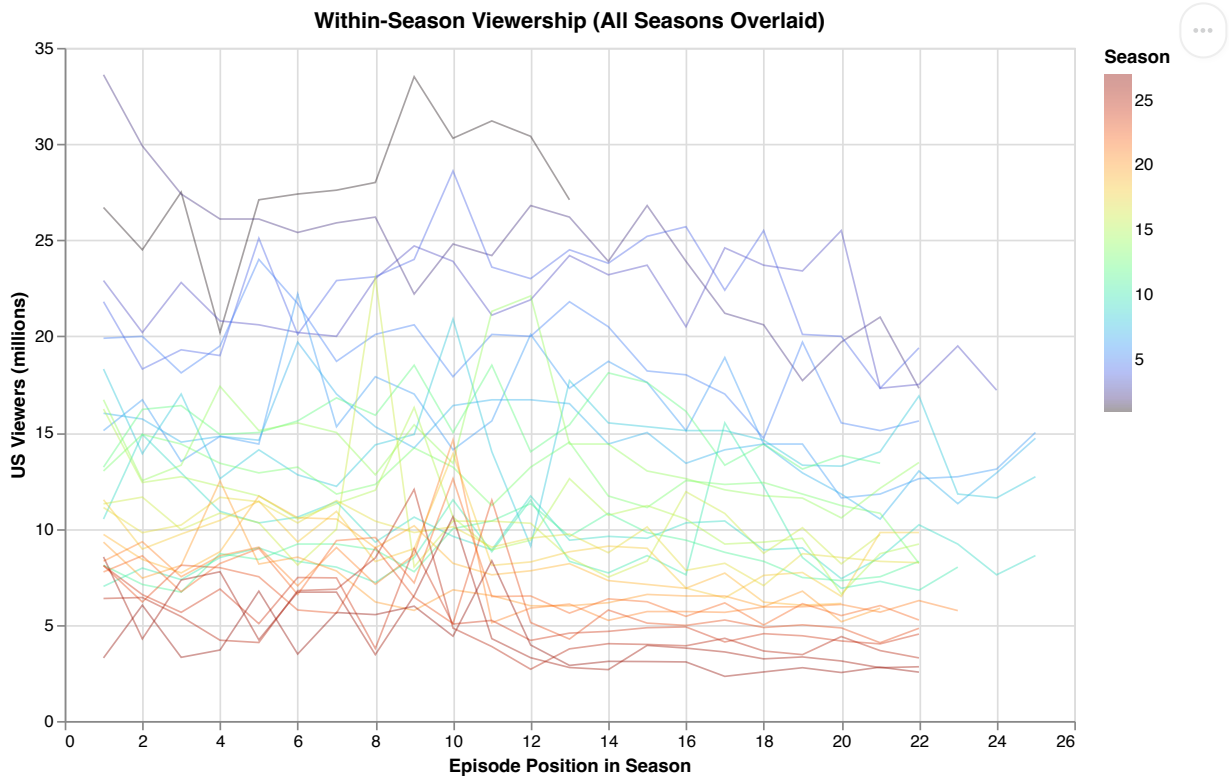
Out[15]:



In [16]:

```
# Overlaid line chart (all seasons)
alt.Chart(df).mark_line(opacity=0.4, strokeWidth=1).encode(
    x=alt.X("number_in_season:Q", title="Episode Position in Season"),
    y=alt.Y("us_viewers_in_millions:Q", title="US Viewers (millions)"),
    color=alt.Color("season:Q", scale=alt.Scale(scheme="turbo"), title="Season"),
    detail="season:N",
    tooltip=["season:O", "number_in_season:O", "us_viewers_in_millions:Q"],
).properties(
    width=600,
    height=400,
    title="Within-Season Viewership (All Seasons Overlaid)",
)
```

Out[16]:

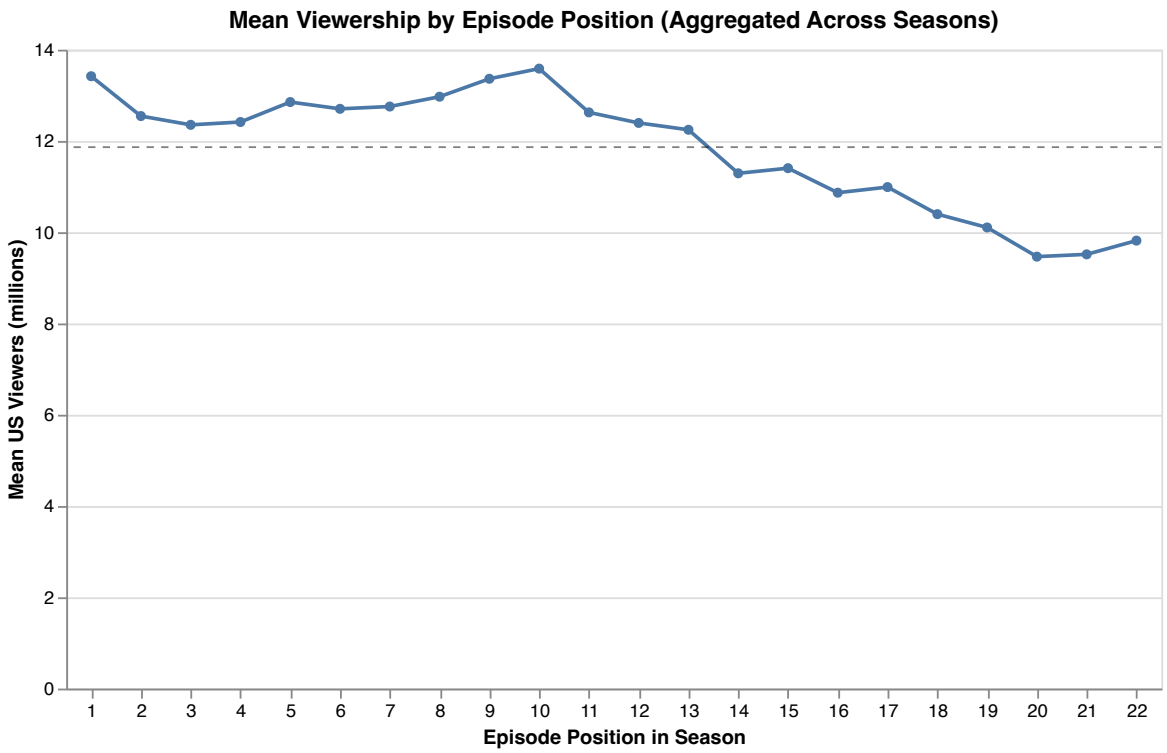


In [17]:

```
# Mean viewership by episode position (aggregated across all seasons)
pos_stats = (
    df.groupby("number_in_season")["us_viewers_in_millions"]
    .agg(["mean", "count"])
    .reset_index()
)
pos_stats = pos_stats[pos_stats["count"] >= 10]

overall_mean = df["us_viewers_in_millions"].mean()
```

```
q5_agg_line = (
    alt.Chart(pos_stats)
    .mark_line(point=True, strokeWidth=2)
    .encode(
        x=alt.X(
            "number_in_season:O",
            title="Episode Position in Season",
            axis=alt.Axis(labelAngle=0),
        ),
        y=alt.Y("mean:Q", title="Mean US Viewers (millions)"),
        color=alt.value("#4c78a8"),
        tooltip=[
            alt.Tooltip("number_in_season:O", title="Episode"),
            alt.Tooltip("mean:Q", title="Mean Viewers (M)", format=".2f"),
            alt.Tooltip("count:Q", title="Seasons with this position"),
        ],
    )
)

q5_agg_ref = (
    alt.Chart(pd.DataFrame({"y": [overall_mean]}))
    .mark_rule(color="black", strokeDash=[4, 4], opacity=0.5)
    .encode(y="y:Q")
)

(q5_agg_line + q5_agg_ref).properties(
    width=600,
    height=350,
    title="Mean Viewership by Episode Position (Aggregated Across Seasons)",
)
```

Out[17]:



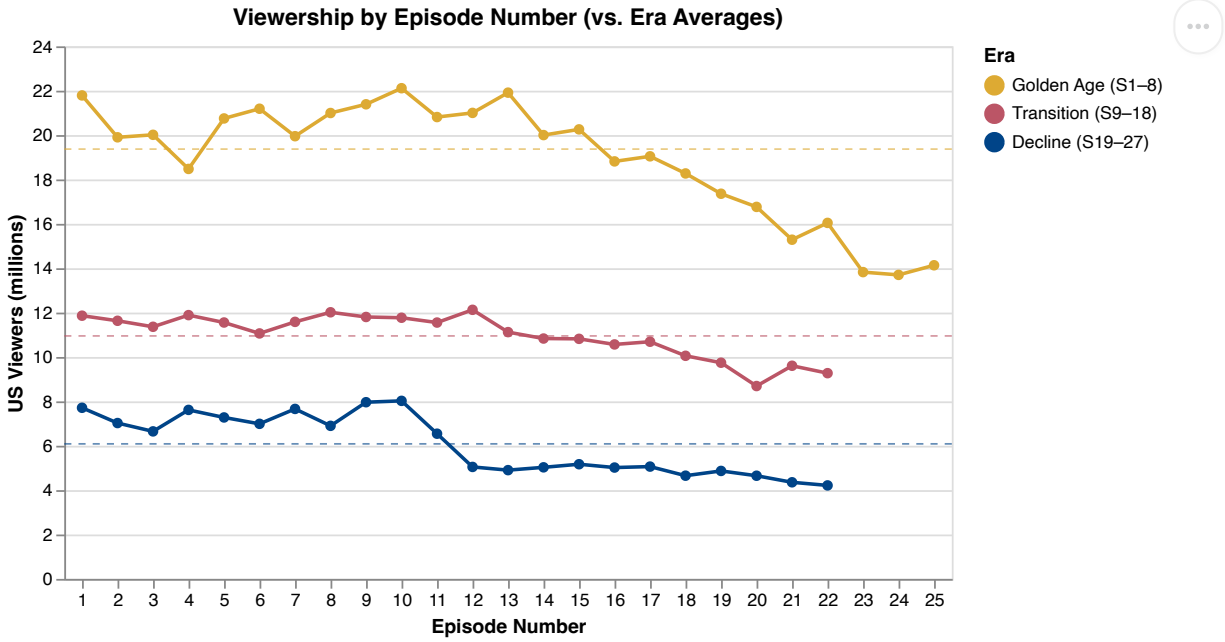
Q5: Final Visualization

In [18]:

```
pos_era_abs = (
    df.groupby(["era", "number_in_season"], observed=True)["us_viewers_in_millions"]
    .agg(["mean", "count"])
    .reset_index()
)
pos_era_abs = pos_era_abs[pos_era_abs["count"] >= 3]
era_avgs = (
    df.groupby("era", observed=True)["us_viewers_in_millions"]
```

```
    .mean()
    .reset_index(name="era_mean")
)

base = alt.Chart(pos_era_abs).encode(
    x=alt.X("number_in_season:O", title="Episode Number", axis=alt.Axis(labelAngle=0)),
    y=alt.Y("mean:Q", title="US Viewers (millions)"),
    color=alt.Color("era:N", scale=era_scale, legend=alt.Legend(title="Era")),
)

q5_lines_points = base.mark_line(
    strokeWidth=2, point=alt.OverlayMarkDef(filled=True, size=35)
).encode(
    tooltip=[
        "era:N",
        "number_in_season:O",
        alt.Tooltip("mean:Q", format=".2f"),
        "count:Q",
    ]
)
q5_refs = (
    alt.Chart(era_avgs)
    .mark_rule(strokeDash=[4, 4], opacity=0.6)
    .encode(y="era_mean:Q", color=alt.Color("era:N", scale=era_scale, legend=None))
)
chart5 = (q5_lines_points + q5_refs).properties(
    height=300, title="Viewership by Episode Number (vs. Era Averages)"
)
chart5
```

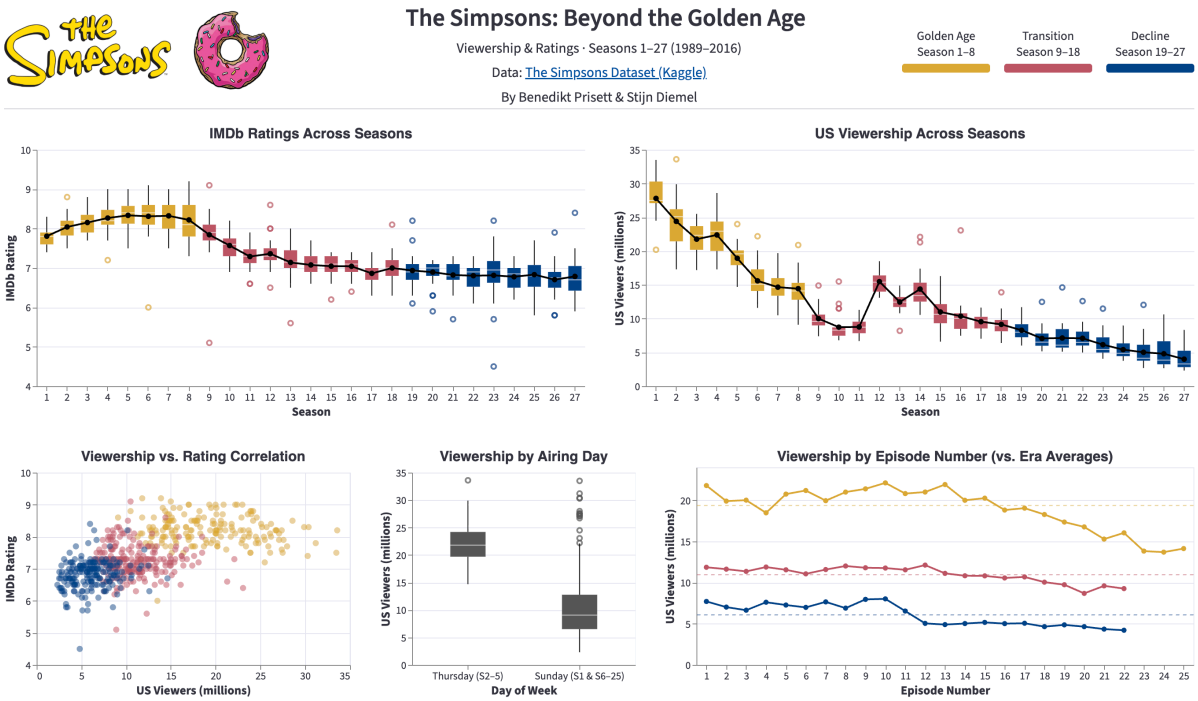
Out[18]:



As question 5 was more open-ended, we first looked at how premiere, finale, and average viewership evolved over time, but quickly became more interested in how an episode's position within a season relates to its viewership numbers. Our initial plot overlaid all individual season viewership lines on a single chart, colored by season on a continuous scale, but with 27 overlapping lines the chart was cluttered and impossible to read. We simplified this into a single aggregated line showing mean viewership by episode position across all seasons, with a dashed overall average as reference. For the final version we split the aggregated line by era to fit the visual story established in the other charts and reveal more detail. The resulting multi-line chart shows mean viewership per episode position for each of the three eras. We added dashed reference lines showing the era averages to intentionally highlight that later episodes within a season tend to drop below their era's average, indicating that early episodes are consistently more popular. A reader can follow any of the

three lines and see the drop-off relative to its dashed reference, making the comparison intuitive and showing that this declining pattern appears across all three eras.

Final Dashboard Visualization



The final visualization combines all five charts in a single static multi-view dashboard. The two box plot charts are placed side by side in the top row, as they address similar questions and share the same style. They are also the most straightforward charts to read, so the viewer immediately grasps the central narrative of the overall decline in popularity. With this context established, the bottom row allows the reader to investigate further. We added a header that provides all relevant context at a glance: a title paired with images to draw the viewer in, key facts such as the data range, source, and authors. Based on the era grouping that emerged from the correlation analysis, we extended this color scheme to all charts except the airing day comparison, creating a consistent visual story connecting the charts and revealing additional, immediately apparent insights. A single shared legend in the header ensures the viewer only needs to learn the color scheme once and can apply it across the entire dashboard. For the Streamlit implementation, we made several adjustments to ensure consistent styling: adjusted the default theme applied by Streamlit, used consistent font sizes and aligned the rating scale ranges across charts.